متد ()doBind به‌منظور تدارک RPCها بایستی یک رابطی برگرداند که معرف متدهایی در شی دور است که کارفرماها برای تعامل با سرویس استفاده می‌کنند. این رابط با نحو معمول جاوا نوشته می‌شود و از زبان تعریف رابط اندروید[1] (AIDL) استفاده می‌کند تا ته‌فایل‌ها[2] را ایجاد کند که به‌عنوان رابط کارفرما برای سرویس‌های دور می‌مانند.

ما در اینجا، به‌اختصار، فرایند لازم برای فراهم‌ساختن یک سرویس دور عمومی به‌نام ()remoteMethod را با استفاده از AIDL و سرویس پیونددهنده مطرح می‌کنیم. رابط سرویس دور به‌صورت زیر است:

```
/* RemoteService.aidl */
interface RemoteService
{
        boolean remoteMethod(int x, double y);
}
```

این فایل به‌صورت RemoteService.aidl نوشته می‌شود. ابزار توسعه‌ی اندروید از آن استفاده خواهد کرد تا یک رابط java. از فایل aidl. تولید کند، همین‌طور یک ته‌کدی که به‌عنوان رابط RPC برای این سرویس عمل می‌کند. کارگزار بایستی رابط تولید شده توسط فایل aidl. را پیاده‌سازی کند و پیاده‌سازی این رابط در زمانی که کارفرما ()remoteMethod را احضار می‌کند فراخوانی خواهد شد.

موقعی که یک کارفرما ()bindService را فراخوانی می‌کند، متد ()onBind روی کارگزار احضار می‌شود و ته‌کد شئ RemoteService را به کارفرما بر می‌گرداند. در ادامه، کارفرما می‌تواند مند دور را به‌صورت زیر احضار کند:

```
RemoteService service;
        . . .
service.remoteMethod(3, 0.14);
```

در داخل، چارچوب پیونددهنده‌ی اندروید به چینش پارامترها رسیدگی می‌کند، پارامترهای چیده شده را میان پردازه‌ها انتقال می‌دهد، و پیاده‌سازی لازم سرویس را احضار می‌کند، همین‌طور، هرگونه مقدار برگشتی را دوباره به پردازه‌ی کارفرما ارسال می‌کند.

۳-۹ خلاصه

- پردازه یک برنامه‌ی در حال اجراست و وضعیت فعالیت جاری یک پردازه با شمارنده‌ی برنامه، در کنار سایر ثبات‌ها مشخص می‌شود.
- چیدمان یک پردازه در درون حافظه با چهار بخش نمایش داده می‌شود: (۱) متن، (۲) داده‌ها، (۳) هیپ، و (۴) پشته.
- همچنان‌که یک پردازه اجرا می‌شود، تغییر حالت می‌دهد. چهار حالت کلی برای یک

1. Android Interface Definition Language (AIDL)
2. Stub files

پردازه وجود دارد که عبارتند از: (۱) آماده، (۲) اجرا، (۳) انتظار، و (۴) خاتمه‌یافته.

- بلاک کنترل پردازه (PCB) یک ساختمان‌داده‌ی هسته است که یک پردازه را در یک سیستم‌عامل نمایش می‌دهد.
- نقش زمان‌بند پردازه، انتخاب یک پردازه‌ی در دسترس برای اجرا بر روی یک CPU می‌باشد.
- سیستم عامل، زمانی که کنترل اجرا را از یک پردازه به پردازه‌ی دیگر منتقل می‌کند یک برگردان‌متن انجام می‌دهد.
- فراخوانی‌های سیستمی ()fork و ()CreateProcess، به‌ترتیب برای ایجاد پردازه‌ها در یونیکس و ویندوز استفاده می‌شوند.
- زمانی که حافظه‌ی مشترک برای ارتباط میان پردازه‌ها استفاده می‌شود، دو (یا چند) پردازه ناحیه‌ی مشترکی از حافظه را به‌اشتراک می‌گیرند. پوزیکس یک API برای حافظه‌ی مشترک فراهم می‌کند.
- دو پردازه ممکن است با تبادل پیغام با یکدیگر با استفاده از عبور پیغام ارتباط برقرار نمایند. سیستم‌عامل Mach از عبور پیغام به‌عنوان شکل اصلی ارتباط بین پردازه‌ای خود استفاده می‌کند. ویندوز شکلی از عبور پیغام را نیز فراهم می‌کند.
- لوله یک مجرایی برای ارتباط دو پردازه فراهم می‌کند. دو شکل از لوله‌ها وجود دارند، معمولی و با نام. لوله‌های معمولی برای ارتباط میان پردازه‌هایی طراحی می‌شوند که یک رابطه‌ی پدر–فرزندی دارند. لوله‌های با نام کلی‌تر هستند و امکان ارتباط چندین پردازه را فراهم می‌کنند.
- سیستم‌های یونیکس، لوله‌های معمولی را از طریق فراخوانی سیستمی ()pipe فراهم می‌کنند. لوله‌های معمولی دارای یک طرف خواندن و یک طرف نوشتن هستند. برای مثال، یک پردازه‌ی والد می‌تواند با استفاده از طرف نوشتن، داده‌ها را به لوله ارسال کند و پردازه‌ی فرزند می‌تواند آن‌ها را از طرف خواندن بخواند. لوله‌های با نام در یونیکس FIFO تعبیر می‌شوند.
- سیستم‌های ویندوز نیز دو شکل از لوله‌ها را فراهم می‌کنند—لوله‌های بی‌نام و با نام. لوله‌های بی‌نام مشابه لوله‌های معمولی یونیکس هستند. آن‌ها یک‌طرفه بوده و روابط پدر–فرزندی را میان پردازه‌های ارتباطی به‌کار می‌برند. لوله‌های با نام، شکل غنی‌تری از ارتباط بین‌پردازه را به‌نسبت نظایر یونیکس، یعنی FIFOها عرضه می‌کنند.
- دو شکل مرسوم ارتباط کارفرما–کارگزار، سوکت‌ها و RPCها هستند. سوکت‌ها به دو پردازه از ماشین‌های متفاوت اجازه می‌دهند تا از طریق یک شبکه در ارتباط باشند. RPCها مفهوم فراخوانی‌های تابع (روال) را به‌شیوه‌ای انتزاعی می‌سازند که یک تابع می‌تواند روی پردازه‌ای دیگر که ممکن است روی ماشین جداگانه‌ای قرار داشته باشد احضار شود.
- سیستم‌عامل اندروید از RPCها به‌صورت شکلی از ارتباط بین‌پردازه‌ای با استفاده از چارچوب پیونددهنده‌اش استفاده می‌کند.

خودآزمایی

۳-۱ با استفاده از برنامه‌ای که در شکل ۳-۳۰ نشان داده می‌شود، توضیح دهید که خروجی خط A چه مقداری خواهد بود.

۳-۲ با احتساب پردازه‌ی والد آغازین، چند پردازه توسط برنامه‌ی نشان داده شده در شکل ۳-۳۱ ایجاد می‌شوند؟

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int value = 5;
int main()
{
        pid_t pid;
        pid = fork();
        if (pid == 0) {/* child process */
                        value += 15;
                        return 0;
                }
                else if (pid > 0) {/* parent process */
                        wait(NULL);
                        printf("PARENT: value = %d",value); /* LINE A */
                        return 0;
                }
}
```

شکل ۳-۳۰ خروجی خط A چه مقداری خواهد بود؟

```
#include <stdio.h>
#include <unistd.h>
int main()
{
        /* fork a child process */
        fork();
        /* fork another child process */
        fork();
        /* and fork another */
        fork();
        return 0;
}
```

شکل ۳-۳۱ چه تعداد پردازه ایجاد می‌شوند؟

۳-۳ نسخه‌های اولیه‌ی سیستم‌عامل موبایل iOS اَپل، هیچ‌گونه ابزاری برای پردازش همروند فراهم نمی‌کنند. در رابطه با سه پیچیدگی که پردازش همروند به یک سیستم‌عامل می‌افزاید بحث کنید.

۳-۴ برخی سیستم‌های کامپیوتری، چندین مجموعه ثبات فراهم می‌کنند. توضیح دهید که اگر در زمان رخداد یک برگردان متن، متن جدید از قبل در یکی از مجموعه ثبات‌ها بار شده باشد چه اتفاقی می‌افتد. اگر به جای این که متن جدید در یک مجموعه ثبات باشد، در داخل حافظه بوده و تمامی مجموعه ثبات‌ها نیز در حال استفاده باشند چه پیش خواهد آمد؟

۳-۵ زمانی که پردازه‌ای با استفاده از عملیات ()fork پردازه‌ی جدیدی را ایجاد می‌کند، کدام یک از گزینه‌های زیر در میان پردازه‌ی والد و پردازه‌ی فرزند مشترک خواهد بود؟

الف- پشته ب- هیپ ج- سگمنت‌های حافظه‌ی مشترک.

۳-۶ معنی «دقیقاً یکبار» را در رابطه با راهکار RPC در نظر بگیرید. آیا الگوریتم پیاده‌ساز این معنی می‌تواند اجرای درستی داشته باشد در صورتی که، پیغام ACK برگشتی به کارفرما به خاطر بروز یک مشکل در شبکه گم شود؟ دنباله‌ی پیغام‌های مبادله شده را تشریح کرده و در این رابطه که، آیا معنی «دقیقاً یکبار» برقرار می‌ماند یا خیر، بحث کنید.

۳-۷ فرض کنید که در یک سیستم توزیع‌شده امکان از کار افتادن کارگزار وجود دارد. چه راهکارهایی لازمند تا تضمین شود که اجرای RPCها معنی «دقیقاً یکبار» را خواهند داشت؟

تمرین‌ها

۳-۸ اَعمالی را که توسط هسته در برگردان متن میان پردازه‌ها انجام می‌شود، توضیح دهید.

۳-۹ یک درخت پردازه مشابه شکل ۳-۷ بنا کنید. برای کسب اطلاعات پردازه در سیستم یونیکس یا لینوکس از فرمان ps –ael استفاده کنید. با استفاده از فرمان man ps اطلاعات بیشتری را در رابطه با فرمان ps به‌دست آورید. مدیر وظیفه در سیستم‌های ویندوز، ID پردازه‌ی والد را فراهم نمی‌کند، اما ابزار ***process monitor***، که از آدرس technet.microsoft.com در دسترس است، یک ابزار درخت پردازه فراهم می‌کند.

۳-۱۰ نقش پردازه‌ی init (یا systemd) را در سیستم‌های یونیکس و لینوکس در رابطه با

ختم پردازه توضیح دهید.

۳-۱۱با احتساب پردازه‌ی والد آغازین، چند پردازه توسط برنامه‌ی نشان داده شده در شکل ۳-۳۲ ایجاد می‌شوند؟

```
#include <stdio.h>
#include <unistd.h>
int main() {
        int i;
        for (i = 0; i < 4; i++)
                fork();
        return 0;
}
```

شکل ۳-۳۲ چه تعداد پردازه ایجاد می‌شوند؟

۳-۱۲ شرایطی را توضیح دهید، خط کدی که در شکل ۳-۳۳ علامت printf(“LINE J”) خورده است، اجرا خواهد شد.

۳-۱۳ با استفاده از برنامه‌ی شکل ۳-۳۴، مقادیر pid را در خطوط A، B، C و D مشخص کنید (فرض کنید که pidهای واقعی والد و فرزند، به ترتیب، ۲۶۰۰ و ۲۶۰۳ هستند).

۳-۱۴ با یک مثال نشان دهید که در شرایطی ممکن است لوله‌های معمولی مناسب‌تر از لوله‌های با نام باشند و مثال دیگری از شرایطی بیاورید که لوله‌های با نام از لوله‌های معمولی مناسب‌ترند.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main() {
        pid_t pid;

        /* fork a child process */
        pid = fork();
        if (pid < 0) { /* error occurred */
                fprintf(stderr, "Fork Failed");
                return 1;
        }
        else if (pid == 0) { /* child process */
                execlp("/bin/ls","ls",NULL);
                printf("LINE J");
        }
        else { /* parent process */
                /* parent will wait for the child to complete */
                wait(NULL);
                printf("Child Complete");
```

```
        }
        return 0;
}
```

شکل ۳-۳۳ تحت چه شرایطی، خروجی «LINE J» نمایش داده می‌شود؟

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
int main(){
pid_t pid, pid1;
        /* fork a child process */
        pid = fork();
        if (pid < 0) { /* error occurred */
                fprintf(stderr, "Fork Failed");
                return 1;
        }
else if (pid == 0) { /* child process */
                pid1 = getpid();
                printf("child: pid = %d", pid); /* A */
                printf("child: pid1 = %d", pid1); /* B */
        }
else { /* parent process */
                pid1 = getpid();
                printf("parent: pid = %d", pid); /* C */
                printf("parent: pid1 = %d", pid1); /* D */
                wait(NULL);
        }
        return 0;
}
```

شکل ۳-۳۴ مقادیر pid چیست؟

۳-۱۵راهکار RPC را در نظر بگیرید. عواقب نامطلوبی را توصیف کنید که می‌تواند از عدم اجبار معانی «حداکثر یکبار» و یا «دقیقاً یکبار» ناشی شوند. استفاده‌های ممکن یک راهکار را که هیچ یک از این معانی را تضمین نمی‌کند، تشریح نماید.

۳-۱۶با استفاده از برنامه‌ی نشان داده شده در شکل ۳-۳۵ توضیح دهید که خروجی خطوط X و Y چه خواهد بود.

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>
#define SIZE 5
int nums[SIZE] = {0,1,2,3,4};
int main(){
        int i;
```

```
    pid_t pid;
    pid = fork();
    if (pid == 0) {
            for (i = 0; i < SIZE; i++) {
            nums[i] *= -i;
            printf("CHILD: %d ",nums[i]); /* LINE X */
            }
    }
    else if (pid > 0) {
            wait(NULL);
    for (i = 0; i < SIZE; i++)
            printf("PARENT: %d ",nums[i]); /* LINE Y */
    }
    return 0;
}
```

شکل ۳-۳۵ خروجی خط X و خط Y چه خواهد بود؟

۳-۱۷ منافع و معایب هر یک از گزینه‌های زیر کدامند؟ هر دو سطح سیستم و برنامه‌نویس را در نظر بگیرید.

الف) ارتباط همگام و ناهمگام ب) بافرسازی خودکار و آشکار

ج) ارسال با کپی و ارسال با مرجع د) پیغام‌های با اندازه‌ی ثابت و متغیر

مسائل برنامه‌نویسی

۳-۱۸ با استفاده از یک سیستم یونیکس یا لینوکس، برنامه‌ای به زبان C بنویسید که یک پردازه‌ی فرزند ایجاد می‌کند (fork()) و در نهایت یک پردازه‌ی نیمه‌جان می‌شود. این پردازه‌ی نیمه‌جان می‌بایست به مدت حداقل ۱۰ ثانیه در سیستم باقی بماند. حالت‌های پردازه می‌تواند با استفاده از فرمان زیر به‌دست آید:

```
ps -l
```

حالت‌های پردازه در زیر ستون S نشان داده می‌شوند؛ پردازه‌های با حالت Z نیمه‌جان‌ها هستند. شناسه‌ی پردازه‌ی (pid) پردازه‌ی فرزند در ستون PID و والد در ستون PPID فهرست می‌شود.

شاید ساده‌ترین راه تعیین این که پردازه‌ی فرزند به واقع نیمه‌جان است، اجرای برنامه‌ی نوشته شده در پس‌زمینه باشد (با استفاده از &) و سپس می‌توان برای تعیین این که آیا فرزند، یک پردازه‌ی نیمه‌جان است یا خیر، فرمان ps –l را اجرا کرد. چون نمی‌خواهید پردازه‌های نیمه‌جان، زیادی در سیستم وجود داشته باشند، لازم دارید که پردازه‌ای را که ایجاد کرده‌اید حذف کنید. ساده‌ترین راه انجام این کار، ختم

پردازه‌ی والد با استفاده از فرمان kill می‌باشد. برای مثال، اگر pid والد، ۴۸۸۴ باشد، دستور زیر را وارد خواهید کرد:

```
kill -9 4884
```

۳-۱۹ یک برنامه‌ی C به‌نام time.c بنویسید که مدت زمان لازم برای اجرای یک فرمان را از خط فرمان معین می‌کند. این برنامه به‌صورت «./time <command>» اجرا خواهد شد و مدت زمان سپری شده برای اجرای فرمان مشخص شده را گزارش خواهد داد. این کار مشمول استفاده از توابع fork() و exec()، همین‌طور تابع gettimeofday() برای تعیین زمان سپری شده می‌باشد. به‌علاوه، لازم است که از دو راهکار متفاوت IPC استفاده شود.

راهبرد کلی این است که یک پردازه‌ی فرزند ایجاد شود که فرمان مشخص شده را اجرا خواهد کرد. با این وجود، پیش از این که فرزند فرمان را اجرا کند، زمان فعلی را ثبت خواهد کرد (که تعبیر ما «زمان شروع» است). پردازه‌ی والد به انتظار خاتمه‌ی پردازه‌ی فرزند خواهد ماند. همین که فرزند خاتمه پیدا کرد، والد زمان جاری را به‌عنوان زمان پایانی ثبت خواهد کرد. اختلاف میان زمان‌های شروع و پایان معرف زمان سپری شده برای اجرای فرمان است. خروجی نمونه‌ی زیر، مدت زمانی را که برای اجرای فرمان ls گزارش شده نشان می‌دهد.

```
./time ls
time.c
time

Elapsed time: 0.25422
```

چون پردازه‌های والد و فرزند، پردازه‌های جداگانه‌ای هستند، نیاز خواهند داشت تا چگونگی اشتراک زمان شروع را بین خودشان هماهنگ کنند. شما دو نسخه از این برنامه را خواهید نوشت که هر یک، یک روش متفاوتی از IPC را نمایش می‌دهند.

در نسخه‌ی اولی، پردازه‌ی فرزند بایستی قبل از فراخوانی exec()، زمان شروع را در ناحیه‌ای از حافظه‌ی مشترک بنویسد. بعد از این که پردازه‌ی فرزند خاتمه پیدا کرد، والد زمان شروع را از حافظه‌ی مشترک خواهد خواند. برای جزئیات بیشتر در رابطه با حافظه‌ی مشترک پوزیکس به بخش ۳-۷-۱ مراجعه کنید. در همان بخش، برنامه‌های جداگانه‌ای برای تولیدکننده و مصرف‌کننده وجود دارد. چون راه‌حل این مسئله تنها به یک برنامه‌ی منفردی نیاز دارد، ناحیه‌ی حافظه‌ی مشترک می‌تواند قبل از این که پردازه‌ی فرزند جدا شود برپا شود که به هر دو پردازه‌ی والد و فرزند امکان می‌دهد تا ناحیه‌ی مشترک حافظه را دست‌یابی کنند.

نسخه‌ی دوم از یک لوله استفاده خواهد کرد. فرزند، زمان شروع را در لوله خواهد نوشت، و والد، آن را به‌دنبال خاتمه‌ی پردازه‌ی فرزند خواهد خواند.

شما از تابع gettimeofday() برای ثبت زمان فعلی استفاده خواهید کرد. به این تابع یک اشاره‌گر به یک شئ struct timeval ارسال می‌شود که شامل دو عضو است: tv_sec و t_usec. این اعضاء معرف تعداد ثانیه‌ها و میکروثانیه‌های سپری شده از ۱ ژانویه‌ی ۱۹۷۰ هستند (معروف به UNIX EPOCH). کد نمونه‌ی زیر، چگونگی استفاده از این تابع را نمایش می‌دهد:

```
struct timeval current;
gettimeofday(&current,NULL);

// current.tv_sec represents seconds
// current.tv_usec represents microseconds
```

برای IPC میان پردازه‌های والد و فرزند، محتویات اشاره‌گر حافظه‌ی مشترک می‌تواند به struct timeval مقدار بگیرد که معرف زمان شروع است. موقعی که لوله‌ها استفاده می‌شوند، یک اشاره‌گر به یک struct timeval می‌تواند در لوله نوشته—و از آن خوانده—شود.

۳-۲۰ **مدیر pid** یک سیستم‌عامل، مسؤول مدیریت شناسه‌های پردازه‌هاست. موقعی که پردازه‌ای برای بار اول ایجاد می‌شود، یک pid یکتا توسط مدیر pid برای آن انتخاب می‌شود. موقعی که پردازه اجرای خود را تکمیل می‌کند، pid به مدیر pid برگردانده شده و ممکن است بعدها دوباره همین pid توسط مدیر انتخاب شود. شناسه‌های پردازه به صورت کامل در بخش ۳-۳-۱ بحث می‌شوند. آنچه که در اینجا از اهمیت بالایی برخوردار است تشخیص این است که بایستی شناسه‌های پردازه یکتا باشند؛ هیچ دو پردازه‌ی فعالی نمی‌توانند pid یکسانی داشته باشند.

با استفاده از ثابت‌های زیر، بازه‌ی مقادیر ممکن pid را شناسایی کنید:

```
#define MIN_PID 300
#define MAX_PID 5000
```

می‌توانید از هر ساختمان‌داده‌ای به انتخاب خود برای نمایش موجود بودن شناسه‌های پردازه استفاده کنید. یک راهبرد این است که از آنچه که لینوکس انجام می‌دهد پیروی کرده و از یک نقشه‌ی بیتی استفاده کنید که در آن، مقدار ۰ مکان i نشان می‌دهد که id پردازه با مقدار i موجود است و مقدار ۱ نشان می‌دهد که id پردازه در حال حاضر استفاده می‌شود.

برای کسب و رهایی یک pid، API زیر را پیاده‌سازی کنید:

- int allocate_map(void)—یک ساختمان‌داده برای نمایش pidها ایجاد و مقدار اولیه می‌دهد؛ با عملیات موفق، ۱ و در غیر این صورت، ۱- بر می‌گرداند.
- int allocate_pid(void)—یک pid تخصیص داده و بر می‌گرداند؛ در صورتی که نتواند یک pid تخصیص بدهد ۱- بر می‌گرداند (تمامی pidها استفاده می‌شوند).

- void release_pid(int pid)یک pid را رها می‌سازد.

این مسأله‌ی برنامه‌نویسی بعدها در فصل ۴ و فصل ۶ تغییر داده خواهد شد.

۳-۲۱ دغدغه‌های حدس Collatz از این است که موقعی که یک عدد صحیح مثبت را برداریم و به الگوریتم زیر اعمال کنیم چه اتفاقی می‌افتد:

$$n = \begin{cases} n/2, & \text{if n is even} \\ 3 \times n + 1, & \text{if n is odd} \end{cases}$$

حدس این است: موقعی که این الگوریتم مداوم اعمال می‌شود، تمامی اعداد صحیح مثبت در نهایت به ۱ می‌رسند. برای مثال، اگر n = 35 باشد، دنباله‌ی زیر تولید می‌شود:

```
35, 106, 53, 160, 80, 40, 20, 10, 5, 16, 8, 4, 2, 1
```

برنامه‌ای به زبان C با استفاده از فراخوانی سیستمی ()fork بنویسید که این دنباله را در پردازه‌ی فرزند تولید کند. عدد شروع از خط فرمان داده خواهد شد. برای مثال، اگر ۸ به عنوان یک پارامتر از طریق خط فرمان ارسال شود، پردازه‌ی فرزند خروجی ۸، ۴، ۲، ۱ را خروجی خواهد نمود. چون پردازه‌های والد و فرزند، کپی داده‌های خاص خود را دارند، ضرورت دارد که فرزند، دنباله را خروجی نماید. والد بایستی پیش از خروج از برنامه، با احضار فراخوانی ()wait، به انتظار تکمیل پردازه‌ی فرزند بماند. بررسی خطای لازم را جهت حصول اطمینان از این که از خط فرمان، عدد صحیح مثبت ارسال می‌شود، انجام دهید.

۳-۲۲ در تمرین ۳-۲۱، پردازه‌ی فرزند بایستی دنباله‌ی اعداد تولید شده از الگوریتم مشخص شده توسط حدس Collatz را خروجی نماید، چون والد و فرزند، کپی داده‌های مخصوص به خود را دارند. رویکرد دیگر طراحی این برنامه، برپایی یک شئ حافظه‌ی مشترک میان پردازه‌های والد و فرزند است. این تکنیک به فرزند امکان می‌دهد که محتوای دنباله را در شئ حافظه‌ی مشترک بنویسد. چون حافظه مشترک است، هرگونه تغییری که فرزند ایجاد کند در پردازه‌ی والد نیز منعکس خواهد شد.

این برنامه، ساختاری مثل حافظه‌ی مشترک پوزیکس که در بخش ۳-۷-۱ مطرح شد، خواهد داشت. پردازه‌ی والد با گذر از مراحل زیر پیشروی خواهد داشت:

الف) برپایی شئ حافظه‌ی مشترک (()shm_open، ftruncate() و ()mmap).

ب) ایجاد پردازه‌ی فرزند و انتظارکشی ختم آن.

ج) خروجی محتوای حافظه‌ی مشترک.

د) حذف شئ حافظه‌ی مشترک.

یک وجه نگران‌کننده به پردازه‌های همکار بر می‌گردد که مشمول ملاحظات همگامی است. در این تمرین، بایستی پردازه‌های والد و فرزند هماهنگ شوند تا این که، مادامی که اجرای فرزند تمام نشده است، والد اقدام به خروجی دنباله ننماید. این دو پردازه با استفاده از فراخوانی سیستمی ()wait همگام می‌شوند: پردازه‌ی والد ()wait را احضار می‌کند که آن را معلق خواهد کرد تا این که پردازه‌ی فرزند خارج شود.

۳-۲۳ بخش ۳-۸-۱ بیان می‌کند که شمارگان درگاه خاصی شناخته شده هستند؛ یعنی، آنها سرویس‌های استانداردی را فراهم می‌کنند. درگاه ۱۷ به سرویس *quote-of-the-day* (نقل قول روز) معروف است. موقعی که کارفرما به درگاه ۱۷ روی کارگزار متصل می‌شود، کارگزار با نقل قولی برای همان روز پاسخ می‌دهد.

کارگزار تاریخ نشان داده شده در شکل ۳-۲۷ را چنان اصلاح کنید که به جای تاریخ جاری، نقل قول روز را ارسال کند. نقل قول‌ها بایستی کاراکترهای ASCII چاپی بوده و باید کمتر از ۵۱۲ کاراکتر باشند، هرچند که چندخط بودن مجاز است. از آنجایی که درگاه‌های شناخته شده رزرو هستند و از این رو در دسترس نیستند، کارگزار شما به درگاه ۶۰۱۷ گوش خواهد داد. کارفرمای تاریخ که در شکل ۳-۲۸ نشان داده می‌شود می‌تواند برای خواندن نقل قول‌های برگشتی کارگزار شما به کار گرفته شود.

۳-۲۴ هایکو[1] یک شعر سه خطی است که خط اول آن شامل پنج هجا، خط دوم شامل هفت هجا و خط سوم شامل پنج هجاست. یک کارگزار هایکو بنویسید که به درگاه ۵۵۷۵ گوش می‌دهد. موقعی که کارفرمایی به این درگاه متصل می‌شود، کارگزار با یک هایکو پاسخ می‌دهد. کارفرمای نشان داده شده در شکل ۳-۲۸ می‌تواند برای خواندن اشعار برگشتی کارگزار هایکو استفاده شود.

۳-۲۵ **کارگزار اِکو**، کارگزاری است که هر چیزی را که دریافت می‌کند دومرتبه برای کارفرما بر می‌گرداند. برای مثال، اگر کارفرما برای کارگزار رشته‌ی «Hello there!» را ارسال کند، کارگزار با «Hello there!» پاسخ خواهد داد.

با استفاده از APIی شبکه در جاوا که در بخش ۳-۸-۱ مطرح شد، یک کارگزار اکو بنویسید. این کارگزار با استفاده از متد ()accept منتظر اتصال یک کارفرما می‌ماند. زمانی که اتصالی از کارفرما می‌رسد، کارگزار در یک حلقه افتاده و گام‌های زیرین را انجام می‌دهد:

- خواندن داده‌های سوکت به داخل یک بافر.
- نوشتن دوباره‌ی محتویات بافر برای کارفرما.

1. Haiku

کارگزار فقط زمانی حلقه را ترک می‌کند که متوجه شود، کارفرما اتصال را بسته است.

کارگزار تاریخ شکل ۳-۲۷ از کلاس java.io.BufferedReader استفاده می‌کند. BufferedReader کلاس java.io.Reader را توسعه می‌دهد که برای خواندن جریان‌های کاراکتری استفاده می‌شود. با این وجود، کارگزار اکو نمی‌تواند تضمین نماید که فقط کاراکترها را از کارفرماها خواهد خواند؛ شاید داده‌های دودویی نیز دریافت کند. کلاس java.io.InputStream با داده‌ها به جای سطح کاراکتری در سطح بایت سر و کار دارد. بنابراین، کارگزار اکو بایستی از شیئی استفاده کند که java.io.InputStream را توسعه می‌دهد. زمانی که کارفرما، سوکت اتصال طرف خود را می‌بندد، متد ()read کلاس java.io.InputStream مقدار ۱- بر می‌گرداند.

۳-۲۶ با استفاده از لوله‌های معمولی برنامه‌ای طراحی کنید که در آن یکی از پردازه‌ها یک پیغام رشته‌ای برای پردازه‌ی دوم می‌فرستد و پردازه‌ی دوم، حالت کوچکی و بزرگی حروف پیغام را معکوس کرده و دومرتبه آن را به پردازه‌ی اول ارسال می‌کند. برای مثال، اگر پردازه‌ی اولی پیغام Hi There را ارسال کرده باشد، پردازه‌ی دوم پیغام hI tHERE را برگشت خواهد داد. لازمه‌ی این کار استفاده از دو لوله می‌باشد که یکی برای ارسال پیغام اصلی از پردازه‌ی اولی به دومی و دیگری برای ارسال پیغام تغییریافته از پردازه‌ی دومی به اولی است. این برنامه را می‌توانید با استفاده از لوله‌های یونیکس و یا ویندوز بنویسید.

۳-۲۷ برنامه‌ای برای کپی‌برداری از یک فایل به‌نام filecopy.c با استفاده از لوله‌های معمولی طراحی کنید. به این برنامه دو پارامتر ارسال می‌شود: نام فایلی که کپی می‌شود و نام فایل مقصد. سپس، برنامه یک لوله‌ی معمولی ایجاد کرده و محتوای فایل کپی‌شونده را در لوله می‌نویسد. پردازه‌ی فرزند، این فایل را از لوله خوانده و در فایل مقصد می‌نویسد. برای مثال، اگر برنامه را به صورت زیر احضار کنیم:

```
./filecopy input.txt copy.txt
```

فایل input.txt در لوله نوشته خواهد شد. پردازه‌ی فرزند، محتویات فایل را از لوله خوانده و در فایل مقصد copy.txt می‌نویسد. این برنامه را می‌توانید با استفاده از لوله‌های یونیکس و یا ویندوز بنویسید.

پروژه‌های برنامه‌نویسی

پروژه‌ی ۱- پوسته‌ی یونیکس

این پروژه شامل طراحی یک برنامه‌ی C می‌باشد که به عنوان یک رابط پوسته عمل کرده و فرمان‌های کاربر را پذیرفته و سپس هر فرمان را در پردازه‌ای مجزا اجرا می‌کند. پیاده‌سازی شما از تغییر جهت ورودی و خروجی پشتیبانی خواهد کرد، همین‌طور لوله‌ها به‌صورت شکلی

از IPC میان یک جفت فرمان. تکمیل این پروژه مشمول استفاده از فراخوانی‌های سیستمی ()fork، ()exec، ()wait، ()dup2، و ()pipe یونیکس خواهد بود و می‌تواند روی هر سیستم لینوکس، یونیکس، یا macOS تکمیل شود.

۱- کلیات

بعد از این که فرمان بعدی وارد می‌شود، رابط پوسته برای کاربر یک اعلان می‌دهد. مثال زیر، اعلان osh> را به تصویر می‌کشد و فرمان بعدی کاربر cat prog.c (این فرمان، فایل prog.c را با استفاده از فرمان cat یونیکس بر روی ترمینال نمایش می‌دهد) می‌باشد.

```
osh> cat prog.c
```

یک تکنیک پیاده‌سازی رابط پوسته این است که پردازه‌ی والدی داشته باشیم که در ابتدا آنچه را که کاربر از خط فرمان وارد می‌کند بخواند (در اینجا، cat prog.c) و سپس، پردازه‌ی فرزند مجزایی ایجاد کند که فرمان را انجام می‌دهد. مگر این که جور دیگری گفته شود، پردازه‌ی والد پیش از ادامه‌ی اجرا منتظر می‌ماند تا فرزند خارج شود. این موضوع مشابه کارکرد ایجاد پردازه‌ی جدیدی است که در شکل ۳-۹ نشان داده شد. با این همه، پوسته‌های یونیکس نوعاً به پردازه‌ی فرزند نیز اجازه می‌دهند تا در پس‌زمینه یا همروند اجرا شود. برای انجام این کار، یک علامت ampersand (&) در انتهای فرمان می‌گذاریم. بنابراین، اگر فرمان بالا را به صورت زیر بازنویسی کنیم:

```
osh> cat prog.c &
```

پردازه‌های والد و فرزند اجرای همروندی خواهند داشت.

پردازه‌ی فرزند جداگانه‌ای با استفاده از فراخوانی سیستمی ()fork ایجاد می‌شود و فرمان کاربر با بکارگیری یکی از فراخوانی‌های سیستمی خانواده‌ی ()exec (چنان که در بخش ۳-۳-۱ بیان شد) اجرا می‌گردد.

برنامه‌ای به زبان C که عملیات کلی یک پوسته‌ی خط فرمان را فراهم می‌کند در شکل ۳-۳۶ مهیا شده است. تابع ()main اعلان osh-> را نمایش داده و گام‌های لازم را که بعد از خواندن ورودی کاربر برداشته می‌شوند، طرح‌ریزی می‌کند. تابع ()main به صورت مداوم، مادامی که should_run برابر ۱ است در حلقه دور می‌زند؛ موقعی که کاربر در خط اعلان exit وارد کرد، برنامه مقدار should_run را ۰ کرده و خاتمه می‌پذیرد.

این پروژه در چند بخش سازمان‌دهی می‌شود:

۱- ایجاد پردازه‌ی فرزند و اجرای فرمان در فرزند
۲- فراهم‌ساختن یک امکان سابقه
۳- افزودن پشتیبانی تغییر جهت ورودی و خروجی
۴- امکان ارتباط پردازه‌های والد و فرزند از طریق یک لوله

۲- اجرای فرمان در یک پردازه‌ی فرزند

اولین کار، اصلاح تابع ()main در شکل ۳-۳۶ است چنان که یک پردازه‌ی فرزند ایجاد شده و

فرمان مشخص شده توسط کاربر را اجرا می‌کند. لازمه‌ی این کار، تفکیک ورودی کاربر به توکن‌های مجزا و ذخیره‌ی آنها در یک آرایه از رشته‌های کاراکتری است (args در شکل ۳-۳۶). برای مثال، اگر کاربر فرمان ps –ael را در خط اعلان <osh وارد کند، مقادیر ذخیره شده در آرایه‌ی args عبارتند از:

```
args[0] = "ps"
args[1] = "-ael"
args[2] = NULL
```

آرایه‌ی args به تابع ()execvp ارسال خواهد شد که دارای prototype زیر است:

```
execvp(char *command, char *params[]);
```

در اینجا، command معرف فرمانی است که قرار است انجام شود و params پارامترهای این فرمان را ذخیره دارد. برای این پروژه، تابع ()execvp بایستی به صورت execvp(args[0], args) احضار شود. مطمئن شوید که بررسی می‌کنید آیا کاربر با گذاشتن & می‌خواهد پردازه‌ی والد به انتظار خروج فرزند بماند یا خیر.

```
#include <stdio.h>
#include <unistd.h>
#define MAX_LINE 80 /* The maximum length command */
int main(void)
{
        char *args[MAX_LINE/2 + 1]; /* command line arguments */
        int should_run = 1;/* flag to determine when to exit program */
        while (should_run) {
        printf("osh>");
        fflush(stdout);
                /**
                * After reading user input, the steps are:
                * (1) fork a child process using fork()
                * (2) the child process will invoke execvp()
                * (3) parent will invoke wait() unless command included
                &
                */
        }
        return 0;
}
```

شکل ۳-۳۶ طرح پوسته‌ی ساده.

۳- ایجاد یک امکان سابقه

کار بعدی، تغییر برنامه‌ی رابط پوسته است طوری که یک امکان ***سابقه*** فراهم کند تا کاربر بتواند اخیرترین فرمان را با واردکردن !! اجرا کند. برای مثال، اگر یک کاربر فرمان ls –l را وارد کند، او در ادامه می‌تواند آن فرمان را دوباره با واردکردن !! در خط اعلان اجرا کند. هر فرمانی

که با این شیوه اجرا شود بایستی به صفحه‌نمایش کاربر منعکس شود و بایستی فرمان به‌عنوان فرمان بعدی در بافر سابقه گذاشته شود.

برنامه‌ی شما بایستی رسیدگی به خطای مبنایی را نیز مدیریت کند. اگر هیچ فرمان اخیری در سابقه نباشد، واردکردن !! بایستی یک پیغام «No commands in history.» نشان بدهد.

۴- تغییر جهت ورودی و خروجی

در ادامه، پوسته‌ی شما بایستی اصلاح شود تا از عملگرهای تغییر جهت «>» و «<» پشتیبانی نماید، که «>» خروجی یک فرمان را به یک فایل تغییر هدایت کرده و «<» ورودی را از یک فایل به یک فرمان هدایت می‌کند. برای مثال، اگر کاربری فرمان زیر را وارد کند

```
osh>ls > out.txt
```

خروجی فرمان ls به فایل out.txt هدایت خواهد شد. با تشابهی، ورودی نیز می‌تواند تغییر جهت داده شود. برای مثال، اگر کاربر فرمان زیر را وارد کند

```
osh>sort < in.txt
```

فایل in.txt به‌عنوان ورودی برای فرمان sort عمل خواهد کرد.

مدیریت تغییر جهت هر دوی ورودی و خروجی مشمول استفاده از تابع dup2() خواهد بود که توصیف‌گر یک فایل موجود را به توصیف‌گر فایل دیگر تکثیر می‌کند. برای مثال، اگر fd یک توصیف‌گر فایلی برای فایل out.txt باشد، فراخوانی

```
osh>sort < in.txt
```

fd را در خروجی استاندارد (ترمینال) تکثیر می‌کند. این بدان معنی است که هر نوشته‌ای در خروجی استاندارد به‌واقع به فایل out.txt ارسال خواهد شد.

می‌توانید فرض کنید که فرمان‌ها دارای یک تغییر جهت ورودی و یا خروجی خواهند بود و هر دو را نخواهند داشت. به‌عبارت دیگر، شما لازم نیست که نگرانی توالی فرمان‌هایی همچون sort < in.txt > out.txt را داشته باشید.

۵- ارتباط از طریق یک لوله

اصلاح آخری در پوسته‌ی شما این است که اجازه دهد خروجی یک فرمان به‌عنوان ورودی به فرمان دیگر با استفاده از یک لوله عمل کند. برای مثال، توالی فرمان زیر

```
osh>ls -l | less
```

باعث می‌شود که خروجی فرمان ls –l به‌عنوان ورودی فرمان less عمل کند. هر دوی فرمان‌های ls و less به‌صورت پردازه‌های جداگانه‌ای اجرا خواهند داشت و با استفاده از تابع pipe() یونیکس، آن‌گونه که در بخش ۳-۷-۴ بیان شد ارتباط خواهند داشت. شاید آسان‌ترین راه ایجاد این پردازه‌های جداگانه این باشد که اجازه دهید پردازه‌ی والد، پردازه‌ی فرزند را ایجاد کند (که فرمان ls –l را اجرا خواهد کرد). این فرزند پردازه‌ی فرزند دیگری ایجاد خواهد کرده (که فرمان less را اجرا خواهد کرد) و خط لوله‌ای میان خود و پردازه‌ی فرزندی که ایجاد می‌کند برپا خواهد نمود. پیاده‌سازی کارکرد لوله مستلزم استفاده از تابع dup2() خواهد بود چنان‌چه در بخش قبلی تشریح شد. در نهایت، اگرچه چندین فرمان می‌تواند با استفاده از لوله‌های متعددی

با همدیگر زنجیر شود، شما می‌توانید فرض کنید که فرمان‌ها صرفاً شامل یک کاراکتر لوله بوده و با هیچ‌گونه عملگر تغییر جهتی ترکیب نخواهند شد.

پروژه‌ی ۲- ماژول هسته‌ی لینوکس برای اطلاعات وظیفه

در این پروژه، یک ماژول هسته خواهید نوشت که از سیستم‌فایل proc/ برای نمایش اطلاعات یک وظیفه بر اساس مقدار شناسه‌ی پردازه‌اش، pid استفاده می‌کند. پیش از شروع این پروژه، مطمئن شوید که پروژه‌ی برنامه‌نویسی ماژول هسته‌ی لینوکس فصل ۲ را تکمیل کرده‌اید که مشمول ایجاد یک وارده‌ای در سیستم‌فایل proc/ می‌باشد. این پروژه شامل نوشتن یک شناسه‌ی پردازه در فایل proc/pid/ می‌باشد. همین که pid در فایل proc/ نوشته شد، خواندنی‌های بعدی از proc/pid/ (۱) فرمانی را که در حال اجراست، (۲) مقدار pid وظیفه، و (۳) حالت فعلی وظیفه را گزارش خواهد داد. یک مثال از این که چگونه ماژول هسته‌ی شما به‌محض این که در سیستم بارگذاری شد، دسترسی خواهد شد به‌صورت زیر است:

```
echo "1395" > /proc/pid
cat /proc/pid
command = [bash] pid = [1395] state = [1]
```

فرمان اِکو کاراکترهای «۱۳۹۵» را در فایل proc/pid/ می‌نویسد. ماژول هسته‌ی شما این مقدار را خواهد خواند و معادل عددی صحیح آن را به‌صورتی که معرف یک شناسه‌ی پردازه باشد ذخیره می‌کند. فرمان cat از proc/pid/ می‌خواند، جایی که ماژول هسته‌ی شما، سه فیلد را از task_struct مرتبط با وظیفه‌ای که مقدار pid آن ۱۳۹۵ است بازیابی خواهد کرد.

۱- نوشتن در سیستم‌فایل /proc

در پروژه‌ی ماژول هسته‌ی فصل ۲ آموختید که چگونه از سیستم‌فایل proc/ بخوانید. ما اکنون چگونگی نوشتن در proc/ را پوشش می‌دهیم. مقداردهی فیلد write. در struct file_operations به صورت زیر

```
.write = proc_write
```

باعث می‌شود که تابع proc_write() شکل ۳-۳۷ موقعی که یک عملیات نوشتن به proc/pid/ صورت می‌گیرد، فراخوانی شود.

تابع kmalloc() معادل هسته‌ی تابع سطح کاربری malloc() برای تخصیص حافظه می‌باشد، به‌استثنای این که آن حافظه‌ی هسته را تخصیص می‌دهد. پرچم GFP_KERNEL تخصیص حافظه‌ی معمول هسته را نشان می‌دهد. تابع copy_from_user() محتویات usr_buf (که شامل آن چیزی است که در proc/pid/ نوشته شده است) را به حافظه‌ی هسته که اخیراً تخصیص یافته کپی می‌کند. ماژول هسته‌ی شما باید معادل صحیح این مقدار را با استفاده از تابع هسته‌ی kstrtol() بدست آورد که امضایی به‌صورت زیر دارد:

```
int kstrtol(const char *str, unsigned int base, long *res)
```

این تابع معادل کاراکتری str را ذخیره می‌کند که در مبنای base با res بیان می‌شود.

در نهایت، توجه داشته باشید که ما حافظه‌ای را که قبلاً با ()kmalloc تخصیص یافته بود با فراخوانی ()kfree به هسته بر می‌گردانیم. دقت در مدیریت حافظه—که شامل رهاسازی حافظه برای جلوگیری از **ریزش حافظه**[1] می‌باشد—در زمان توسعه‌ی کد سطح هسته بسیار مهم است.

```
ssize_t proc_write(struct file *file, char __user *usr_buf,
        size_t count, loff_t *pos)
{
        int rv = 0;
        char *k_mem;

        /* allocate kernel memory */
        k_mem = kmalloc(count, GFP_KERNEL);

        /* copies user space usr_buf to kernel memory */
        copy_from_user(k_mem, usr_buf, count);

        printk(KERN_INFO "%s\n", k_mem);
        /* return kernel memory */
        kfree(k_mem);
        return count;
}
```

شکل ۳-۳۷ تابع ()proc_write.

۲- خواندن از سیستم‌فایل /proc

همین که شناسه‌ی پردازه ذخیره شد، هرگونه خواندن از proc/pid/، نام فرمان، شناسه‌ی پردازه‌اش، و حالت آن را بر می‌گرداند. همان‌گونه که در بخش ۳-۱ نشان داده می‌شود، PCB در لینوکس با ساختار task_struct نمایش داده می‌شود که در فایل include/، <linux/sched> پیدا می‌شود. با داشتن یک شناسه‌ی پردازه، تابع ()pid_task ساختار مرتبط task_struct را بر می‌گرداند. امضای این تابع به‌صورت زیر است:

```
struct task_struct pid_task(struct pid *pid, enum pid_type type)
```

تابع هسته‌ی (find_vpid(int pid می‌تواند برای بدست آوردن struct pid استفاده شود، و

1. Memory leaks

PIDTYPE_PID می‌تواند به‌عنوان pid_type استفاده شود.

برای یک pid معتبر در سیستم، pid_task، ساختار task_struct آن را بر می‌گرداند. پس از آن می‌توانید مقادیر فرمان، pid، و حالت را نمایش بدهید. (شما احتمالاً مجبور خواهید شد که ساختار task_struct را در <linux/sched.h> بخوانید تا اسامی این فیلدها را بدست آورید.)

اگر pid معتبری به pid_task() ارسال نشود، NULL بر می‌گرداند. مطمئن شوید که در بررسی مناسب خطاها، بررسی این شرط را انجام می‌دهید. اگر این حالت پیش بیاید، تابع ماژول هسته‌ی مرتبط با خواندن proc/pid/ بایستی ۰ برگرداند.

ما در دانلود کدهای اصلی کتاب، برنامه‌ی pid.c به‌زبان C را می‌دهیم که برخی از بلاک‌های سازنده‌ی اساسی شروع این پروژه را فراهم می‌کند.

پروژه‌ی ۳- ماژول هسته‌ی لینوکس برای لیست‌کردن وظایف

شما در این پروژه یک ماژول هسته خواهید نوشت که تمامی وظایف جاری را در یک سیستم لینوکس لیست می‌کند. شما وظایف را هم به‌صورت خطی و هم به‌صورت اول عمق پیمایش خواهید کرد.

۱- پیمایش وظایف به‌صورت خطی

در هسته‌ی لینوکس، ماکروی for_each_process() به‌راحتی امکان پیمایش تمامی وظایف جاری در سیستم را ممکن می‌سازد:

```
#include <linux/sched.h>

struct task_struct *task;

for_each_process(task) {
        /* on each iteration task points to the next task */
}
```

در ادامه، می‌توان فیلدهای مختلف task_struct را همچنان که برنامه، ماکروی for_each_process() را تکرار می‌کند، نمایش داد.

تمرین

یک ماژول هسته طراحی کنید که با استفاده از ماکروی for_each_process() تمامی وظایف سیستم را پیمایش می‌کند. به‌طور خاص، فرمان وظیفه، حالت و id پردازه‌ی هر وظیفه را خروجی نمایید. (احتمالاً، مجبور خواهید شد که ساختار task_struct موجود در <linux/sched.h> را بخوانید تا اسامی این فیلدها را به‌دست آورید.) این کد را در نقطه‌ی ورود ماژول بنویسید تا محتوای آن در بافر سابقه‌ی هسته ظاهر شود که با استفاده از فرمان dmesg قابل رؤیت است. به منظور تصدیق این که کد شما درست کار می‌کند، محتوای بافر سابقه‌ی هسته را با خروجی فرمان زیر که تمامی وظایف سیستم را فهرست می‌کند، مقایسه کنید:

```
ps -el
```

دو مقدار بایستی خیلی شبیه هم باشند. با این وجود، چون وظیفه‌ها پویا هستند، این امکان وجود دارد که یک سری وظیفه در یک فهرست باشند اما در دیگری نباشند.

۲- پیمایش وظایف با یک درخت جستجوی اول عمق

بخش دوم این پروژه شامل پیمایش تمامی وظایف سیستم با استفاده از یک درخت جستجوی اول عمق (DFS) می‌باشد. (به عنوان یک مثال: پیمایش DFS پردازه‌های شکل ۳-۷ برابر ۱، ۸۴۱۵، ۸۴۱۶، ۹۲۹۸، ۹۲۰۴، ۲۸۰۸، ۳۰۲۸، ۳۶۱۰، ۴۰۰۵ است.)

لینوکس درخت پردازه‌اش را به صورت یک سری لیست نگه می‌دارد. با بررسی task_struct در <linux/sched.h> دو شئ struct list_head را مشاهده می‌کنیم: children و sibling.

این اشیاء اشاره‌گرهایی به لیستی از فرزندان وظیفه، همین‌طور همزادهای آن هستند. لینوکس ارجاعی نیز به وظیفه‌ی آغازین در سیستم—init_task—که از نوع task_struct می‌باشد، نگه می‌دارد. با استفاده از این اطلاعات، در کنار عملیات ماکروی روی لیست‌ها می‌توانیم فرزندان init_task را به صورت زیر پیمایش کنیم:

```
struct task_struct *task;
struct list_head *list;
list_for_each(list, &init_task->children) {
       task = list_entry(list, struct task_struct, sibling);
       /* task points to the next child in the list */
}
```

دو پارامتر که هر دو از نوع struct list_head هستند به ماکروی list_for_each() ارسال می‌شود:

- اشاره‌گری به سر لیستی که قرار است پیمایش شود.
- اشاره‌گری به گره سر لیستی که قرار است پیمایش شود.

پارامتر اول در هر تکرار list_for_each()، برابر ساختار لیست فرزند بعدی انتخاب می‌شود. در ادامه، این مقدار را برای دستیابی به هر ساختار در لیست با استفاده از ماکروی list_entry() به کار می‌بریم.

تمرین

با شروع از وظیفه‌ی init_task یک ماژول هسته طراحی کنید که تمامی وظایف سیستم را با استفاده از یک درخت DFS پیمایش کند. درست همانند بخش اول این پروژه، نام، حالت و pid هر یک از وظیفه‌ها را خروجی نمایید. این پیمایش را در ماژول ورود هسته انجام دهید تا این که خروجی آن در بافر سابقه‌ی هسته ظاهر شود.

اگر تمامی وظایف سیستم را خروجی نمایید، وظایف بسیار بیشتری از آنچه که با فرمان ps -ael می‌بینید، خواهید دید. دلیل آن این است که برخی نخ‌ها به عنوان فرزند ظاهر می‌شوند اما به صورت پردازه‌های معمولی حاضر نمی‌شوند. بنابراین، برای بررسی خروجی درخت DFS از فرمان زیر استفاده کنید:

```
ps -eLf
```

این فرمان تمامی وظایف سیستم را–اعم از نخ‌ها–لیست می‌کند. به‌منظور تصدیق این که به واقع یک پیمایش DFS درست انجام داده‌اید، مجبور خواهید شد که روابط مابین خروجی وظایف مختلف را با فرمان ps بررسی کنید.

پروژه‌ی ۴- ساختمان‌داده‌های هسته

ما در بخش ۱-۹، ساختمان‌داده‌هایی را که در سیستم‌عامل‌ها رایج هستند پوشش دادیم. هسته‌ی لینوکس چندین ساختار از این‌ها را فراهم می‌کند. در اینجا، ما استفاده از لیست پیوندی چرخشی دوطرفه را بررسی می‌کنیم که در دسترس توسعه‌دهندگان هسته می‌باشد. بیشتر آنچه که ما بحث می‌کنیم در کد اصلی لینوکس در دسترس قرار دارد—در این نمونه، فایل include، <linux/list.h>—و توصیه می‌کنیم که این فایل را همچنان که مراحل بعدی را دنبال می‌کنید بررسی کنید.

در آغاز، بایستی یک struct تعریف کنید که شامل عناصری است که قرار است در لیست پیوندی درج شوند. ساختار C زیر، رنگ را به‌صورت ترکیبی از قرمز، آبی، و سبز تعریف می‌کند:

```
struct color {
        int red;
        int blue;
        int green;
        struct list head list;
};
```

به عضو struct list_head list توجه کنید. ساختار list_head در فایل include، <linux/types.h> تعریف می‌شود، و هدف آن تعبیه‌کردن لیست پیوندی در میان گره‌هایی است که لیست را تشکیل می‌دهند. این ساختار list_head کاملاً ساده است—صرفاً دو عضو نگه می‌دارد، next و prev، که به وارده‌های بعدی و قبلی در لیست اشاره می‌کنند. لینوکس با تعبیه‌کردن لیست پیوندی در میان ساختار، مدیریت ساختمان‌داده را با یک سری توابع *ماکرو* امکان‌پذیر می‌سازد.

۱- درج عناصر در لیست پیوندی

می‌توانیم یک شئ list_head تعریف کنیم که به‌عنوان مرجعی برای سر لیست با استفاده از ماکروی ()LIST_HEAD استفاده می‌کنیم:

```
static LIST_HEAD(color list);
```

این ماکرو متغیر color_list را تعریف و مقداردهی می‌نماید که از نوع struct list_head می‌باشد.

ما نمونه‌های struct color را به‌صورت زیر ایجاد و مقداردهی می‌کنیم:

```
struct color *violet;
```

```
violet = kmalloc(sizeof(*violet), GFP_KERNEL);
violet->red = 138;
violet->blue = 43;
violet->green = 226;
INIT_LIST_HEAD(&violet->list);
```

تابع kmalloc() معادل هسته‌ی تابع سطح کاربری malloc() برای تخصیص حافظه است، به‌استثنای این که در آن، حافظه‌ی هسته تخصیص داده می‌شود. پرچم GFP_KERNEL تخصیص حافظه‌ی معمول هسته را نشان می‌دهد. ماکروی INIT_LIST_HEAD() عضو list را در struct color مقداردهی می‌نماید. ما در ادامه می‌توانیم با استفاده از ماکروی list_add_tail()، این نمونه را به انتهای لیست پیوندی اضافه کنیم:

```
list_add_tail(&violet->list, &color_list);
```

۲- پیمایش لیست پیوندی

پیمایش لیست مشمول استفاده از ماکروی list_for_each_entry() می‌باشد که سه پارامتر می‌پذیرد:

- اشاره‌گری به ساختاری که پیمایش می‌شود
- اشاره‌گری به سر لیستی که پیمایش می‌شود
- نام متغیری که شامل ساختار list_head می‌باشد

کد زیر این ماکرو را نشان می‌دهد:

```
struct color *ptr;
list_for_each_entry(ptr, &color_list, list) {
        /* on each iteration ptr points */
        /* to the next struct color */
}
```

۳- حذف عناصری از لیست پیوندی

حذف عناصری از لیست مشمول استفاده از ماکروی list_del() می‌باشد که به آن، اشاره‌گری به struct list_head ارسال می‌شود:

```
list_del(struct list_head *element);
```

این ماکرو element را از لیست حذف می‌کند در حالی که ساختار باقی لیست را نگه می‌دارد.

شاید ساده‌ترین رویکرد حذف تمامی عناصر از یک لیست پیوندی، حذف هر یک از عناصر در زمان پیمایش لیست باشد. ماکروی list_for_each_entry_safe() بیشتر شبیه به list_for_each_entry() رفتار می‌کند به‌استثنای این که، یک آرگومان اضافی به آن ارسال می‌شود که مقدار اشاره‌گر next آیتم حذفی را نگه می‌دارد. (این امر برای حفظ ساختار لیست ضرورت دارد.) مثال کد زیر این ماکرو را نشان می‌دهد:

```
struct color *ptr, *next;
list_for_each_entry_safe(ptr,next,&color_list,list) {
        /* on each iteration ptr points */
        /* to the next struct color */
        list_del(&ptr->list);
        kfree(ptr);
}
```

توجه کنید که بعد از حذف هر عنصر، ما حافظه‌ای را که قبلاً با kmalloc() تخصیص یافته بود با فراخوانی kfree() دوباره به هسته بر می‌گردانیم.

بخش ۱—تمرین

در نقطه‌ی ورود ماژول، یک لیست پیوندی شامل چهار عنصر struct color ایجاد کنید. لیست پیوندی را پیمایش کرده و محتویات آن را در بافر سابقه‌ی هسته خروجی نمایید. فرمان dmesg را احضار کنید تا به‌محض این که ماژول هسته بارگذاری شد، مطمئن شوید لیست به‌درستی ساخته شده است.

در نقطه‌ی خروج ماژول، عناصر را از لیست پیوندی حذف کرده و حافظه‌ی آزاد را دوباره به هسته برگردانید. باز هم، فرمان dmesg را احضار کنید تا بررسی کنید، به‌محض این که ماژول هسته برداشته می‌شود، لیست حذف می‌شود.

بخش ۲- ارسال پارامتر

این بخش از پروژه مشمول ارسال یک پارامتر به یک ماژول هسته است. ماژول از این پارامتر به‌عنوان مقدار اولیه استفاده کرده و دنباله‌ی Collatz را به‌صورتی که در تمرین ۳-۲۱ بیان شده است تولید خواهد کرد.

ارسال یک پارامتر به یک ماژول هسته

پارامترها می‌توانند موقعی که ماژول‌های هسته بارگذاری می‌شوند به آن‌ها ارسال شوند. برای مثال، اگر نام ماژول هسته collatz باشد، می‌توانیم مقدار اولیه‌ی ۱۵ را به پارامتر start هسته به‌صورت زیر ارسال کنیم:

```
sudo insmod collatz.ko start=15
```

در ماژول هسته، start را با استفاده از کد زیر به‌عنوان یک پارامتر تعریف می‌کنیم:

```
#include<linux/moduleparam.h>
static int start = 25;
module_param(start, int, 0);
```

ماکروی module_param() برای گذاشتن متغیرها به‌عنوان پارامترهای ماژول‌های هسته استفاده می‌شود. سه آرگومان برای module_param() فراهم می‌شود که عبارتند از: (۱) نام پارامتر، (۲)

نوع آن، و (۳) مجوزهای فایل. از آنجایی که ما از یک فایل‌سیستم برای دسترسی پارامتر استفاده نمی‌کنیم، ما دغدغه‌ی مجوزها را نداریم و مقدار پیش‌فرض ۰ را استفاده می‌کنیم. توجه کنید که نام پارامتر استفاده شده با فرمان insmod بایستی با نام پارامتر مرتبط هسته تطابق داشته باشد. در نهایت، اگر در طی بارگذاری با insmod، مقداری برای پارامتر ماژول نگذاریم، مقدار پیش‌فرض (که در اینجا ۲۵ است) استفاده می‌شود.

بخش ۲—تمرین

یک ماژول هسته به‌نام collatz طراحی کنید که یک مقدار اولیه به‌عنوان پارامتر ماژول به آن ارسال می‌شود. ماژول شما در ادامه زمانی که بارگذاری می‌شود، دنباله‌ی مد نظر را در یک لیست پیوندی هسته تولید و ذخیره خواهد کرد. همین که دنباله ذخیره شد، ماژول شما لیست را پیمایش کرده و محتویات آن را در بافر سابقه‌ی هسته خروجی خواهد نمود. از فرمان dmesg استفاده کنید تا به‌محض این که ماژول بارگذاری می‌شود، مطمئن شوید که دنباله به‌درستی تولید می‌شود.

در نقطه‌ی خروجی ماژول، محتویات لیست را حذف کرده و حافظه‌ی آزاد شده را دوباره به هسته برگردانید. باز هم، از dmesg استفاده کنید تا بررسی کنید، به‌محض این که ماژول هسته برداشته می‌شود، لیست حذف می‌گردد.